

1. The History of Python

Python's story begins in the late 1980s in the Netherlands.

- **The Creator:** Python was conceived by **Guido van Rossum** at the Centrum Wiskunde & Informatica (CWI). He began implementing it in December 1989.
- **The Inspiration:** Van Rossum was looking for a "hobby" project to keep him busy around Christmas. He designed Python as a successor to the ABC programming language, aiming to create something that was powerful yet easy to read and use.
- **The Name:** Contrary to popular belief, Python is **not** named after the snake. Van Rossum was a big fan of the British comedy series, **Monty Python's Flying Circus**, and chose the name to reflect the light-hearted and fun nature he wanted the language to embody.
- **The Philosophy:** In 1999, Van Rossum formalized his goals for the language, which centered on making the code "as understandable as plain English" and "suitable for everyday tasks, allowing for short development times." This philosophy is famously captured in *The Zen of Python*, a set of 19 guiding principles for writing Python code.

2. The Advantages of Python

Python's design offers significant benefits, especially for beginners:

Advantage	Description
Readability & Simplicity	Python's syntax is nearly identical to English, using plain keywords and extensive indentation (which is mandatory). This clarity makes the code easier to write, read, and maintain.
Rapid Development	Since Python requires significantly fewer lines of code to achieve the same result compared to languages like C++ or Java, development cycles are much faster. This is excellent for prototyping.
Massive Ecosystem	Python boasts the world's largest collection of third-party modules and libraries (e.g., NumPy, Pandas, Requests). This means someone has likely already solved the problem you are facing, saving you time.
compiled+interpreted	Python code is executed line by line,

	making debugging and testing easier. Developers can immediately see the results of their code changes.
Cross-Platform	A Python program written on a Windows machine will generally run without modification on Linux or macOS, ensuring high portability.

3. Why Python Became So Popular

Python's rise to being one of the world's most popular languages stems from its convergence with two major technological trends:

- **The Data Science Boom:** As data and Machine Learning (ML) became central to business, Python was perfectly positioned to dominate. Its foundational libraries like **NumPy** (for array manipulation), **Pandas** (for data analysis), and frameworks like **TensorFlow** and **PyTorch** (for AI/ML) became the industry standard.
- **Simplicity and Entry Barrier:** Because of its simple syntax, Python is widely taught as the first programming language in universities and coding bootcamps globally, ensuring a constant supply of new developers.

4. What Python Can Create and Where It's Used

Python is a truly general-purpose language, meaning it is not limited to one domain.

A. Major Application Areas (What It Creates)

1. **Data Science & Machine Learning (ML):** Building predictive models, training AI algorithms, and analyzing massive datasets.
2. **Web Development (Backend):** Powering the server side of web applications using frameworks like **Django** and **Flask**. (Examples: Instagram, Spotify).
3. **Automation & Scripting:** Writing scripts to automate repetitive tasks like file organization, renaming thousands of files, scraping data from websites, or sending automated emails.
4. **Software Testing:** Automating quality assurance (QA) and testing procedures for other software applications.
5. **Desktop GUI Applications:** Building desktop applications with frameworks like Tkinter or PyQt.

B. Real-World Users (Where It's Used)

Python is the backbone of technology giants across every sector:

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

Company	Use Case
Google	Uses Python across its infrastructure, search engine, and core AI projects.
Netflix	Uses Python for data science, optimizing its recommendation engine, and backend streaming services.
Spotify	Uses Python for its backend services, including user data analysis and personalized music recommendations.
NASA	Uses Python for scientific computing, simulating spacecraft, and data processing.
Dropbox	The entire desktop client, server logic, and API are heavily dependent on Python.

Python Core Concepts: Variables and Operators

This document provides a detailed breakdown of variables and operators, two of the most fundamental concepts in Python programming.

1. Python Variables: The Containers of Data

A variable in Python is essentially a named storage location that holds a value. Unlike some other languages, Python is **dynamically typed**, meaning you do not need to explicitly

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

declare the variable type (like `int`, `string`, etc.) before using it. Python figures it out automatically based on the value you assign.

The Assignment Process

Variables are created the moment you first assign a value to them using the **assignment operator** (`=`).

```
# 'name' is a variable holding a string value
name = "Amar Panchal"
```

```
# 'age' is a variable holding an integer value
age = 17
```

```
# You can change the value and even the type later (though this is often discouraged)
age = "Seventeen" # 'age' now holds a string
```

Type Checking

To check what type of data a variable is currently storing, you can use the built-in `type()` function:

```
pi_value = 3.14159
print(type(pi_value))
# Output: <class 'float'>
```

2. Rules for Creating Variables (Identifiers)

The names you choose for your variables are called **identifiers**. Python has strict rules regarding what constitutes a valid identifier.

A. Mandatory Naming Rules

A variable name **MUST** adhere to the following rules:

1. **Start Character:** It must start with a letter (A-z) or an underscore (`_`). It **cannot** start with a number.
 - *Valid:* `user_name`, `_system_var`
 - *Invalid:* `1st_name`, `7days`

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

2. **Allowed Characters:** It can only contain alpha-numeric characters (A-z, 0-9) and underscores (_).
 - *Valid:* `total_sales_2024`
 - *Invalid:* `total-sales` (hyphens are subtraction)
3. **Reserved Keywords:** It cannot be a reserved Python keyword (e.g., `for`, `while`, `if`, `class`, `def`, `return`, `True`, `False`).

B. Naming Conventions (Best Practices)

While the rules above dictate validity, these conventions dictate professional readability (often referred to as PEP 8 guidelines):

1. **Readability (Snake Case):** Use lowercase letters, with words separated by underscores. This is the standard for almost all Python variables and functions.
 - *Good:* `student_score`, `is_active`
 - *Avoid:* `studentScore` (Camel Case, common in Java/JavaScript)
2. **Constants:** Variables whose values should not change (like or maximum attempts) are usually written in **all capital letters** with underscores.
 - *Example:* `MAX_ATTEMPTS = 5`, `PI = 3.14159`
3. **Meaningful Names:** Variable names should clearly convey their purpose. Avoid single-letter names unless they are standard in a loop (`i`, `j`).
 - *Good:* `calculate_total`
 - *Avoid:* `calc_t`

3. Python Operators: How to Manipulate Data

Operators are special symbols used to perform operations on variables and values (operands). Python groups these operators into several categories.

A. Arithmetic Operators (Math)

Used for performing mathematical calculations.

Operator	Name	Example	Result
+	Addition	10+5	15
-	Subtraction	10-5	5

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

*	Multiplication	10*5	50
/	Division	10/3	3.333... (Always float)
//	Floor Division	10//3	3 (Rounds down to the nearest whole number)
%	Modulus	10%3	1 (Remainder after division)
**	Exponentiation	2**3	8 (2 ³)

Python Operator Precedence: The PEMDAS Rule

Operator Precedence dictates the hierarchy of operations in a mathematical expression. If an expression contains multiple operators, Python follows standard mathematical rules to decide which operation to execute first.

1. The Precedence Hierarchy (PEMDAS/BODMAS)

Python follows this order from highest priority (executed first) to lowest priority (executed last):

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

Priority (Order)	Acronym Part	Description	Python Operators
1 (Highest)	Parentheses / Brackets	Operations inside parentheses are always executed first, overriding all other rules.	()
2	Exponents / Orders	Exponentiation (raising to a power) is the second highest priority.	**
3	Multiplication and Division	Multiplication, standard Division, Floor Division, and Modulus all share the same priority.	*, /, //, %
4 (Lowest)	Addition and Subtraction	Addition and Subtraction share the lowest priority.	+, -

Example Breakdown

Consider the expression: `result = 10 + 2 * (15 / 3) ** 2`

1. Parentheses (P/B): The operation inside the parentheses is .
 - `15/3=5.0`
 - Expression becomes: `10 + 2 * (5.0) ** 2`
2. Exponents (E/O): Next, the exponentiation is calculated.
 - `5.0**2=25.0`
 - Expression becomes: `10 + 2 * 25.0`
3. Multiplication (M/D): Multiplication is calculated next.
 - `2*25.0=50.0`
 - Expression becomes: `10 + 50.0`

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

4. Addition (A/S): Finally, addition is calculated.

- $10+50.0=60.0$
- Final Result: 60.0

2. Associativity: The Tie-Breaker Rule

When two or more operators of the *same* precedence level appear in an expression (e.g., both multiplication and division, or multiple subtractions), Python uses a tie-breaker rule called Associativity.

A. Left-to-Right Associativity (Standard)

Most operators in Python (including $*$, $/$, $//$, $\%$, $+$, $-$) are left-associative. They are executed from left to right as they appear in the expression.

Example: $12 / 3 * 2$

1. Division and Multiplication have the same priority (Level 3).
2. Python executes from left to right:
 - $12/3=4.0$
 - $4.0*2=8.0$
 - Result: 8.0

B. Right-to-Left Associativity (Exponents Only)

The Exponentiation operator ($**$) is a special case; it is right-associative. This means Python evaluates it from right to left.

Example: $2**3**2$

1. The expression contains two exponentiation operators (Level 2).
2. Python executes from right to left:
 - $3**2=9$
 - Expression becomes: $2**9$
 - $2**9=512$
 - Result: 512

Note: If it were left-associative, the result would be $(2**3)**2=8**2=64$.

3. Best Practice: Use Parentheses ()

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

While understanding the rules is essential, the best practice for professional and readable code is to always use parentheses to clarify the intended order of operations, even if the rules technically cover it.

Example: Ambiguous vs. Clear

Code Style	Readability	Evaluation Order
Ambiguous: $a + b * c$	Low: Requires knowledge of precedence.	b and c multiplied first.
Clear: $(a + b) * c$	High: Parentheses force addition first.	a and b added first.
Clear: $a + (b * c)$	High: Parentheses explicitly confirm multiplication first.	b and c multiplied first.

B. Comparison (Relational) Operators

Used to compare two values. They always return a **Boolean** result (**True** or **False**).

Operator	Name	Example	Result (if x=5,y=10)
$==$	Equal to	$x==5$	True
$!=$	Not equal to	$x!=5$	False

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

>	Greater than	y>x	True
<	Less than	y<x	False
>=	Greater than or equal to	x>=5	True
<=	Less than or equal to	y<=5	False

C. Assignment Operators

Used to assign values to variables. They combine an operation and an assignment into one concise statement.

Operator	Example	Equivalent to
=	x=5	Assigns the value 5 to x
+=	x+=3	x=x+3
-=	x-=3	x=x-3
=	x=3	x=x*3
/=	x/=3	x=x/3

D. Logical Operators

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

Used to combine conditional statements (Boolean values).

Operator	Name	Example	Description
<code>and</code>	Logical AND	<code>x>5 and x<10</code>	Returns <code>True</code> if both statements are <code>True</code> .
<code>or</code>	Logical OR	<code>x>5 or x<4</code>	Returns <code>True</code> if at least one statement is <code>True</code> .
<code>not</code>	Logical NOT	<code>not (x>5)</code>	Reverses the result; returns <code>False</code> if the condition is <code>True</code> .

E. Identity Operators

Used to compare the memory locations of two objects. They check if two variables refer to the *exact same object* in memory.

Operator	Name	Example	Description
<code>is</code>	Is the same object	<code>x is y</code>	Returns <code>True</code> if both variables point to the same object.
<code>is not</code>	Is not the same object	<code>x is not y</code>	Returns <code>True</code> if both variables do not point to the same object.

F. Membership Operators

Used to test if a sequence (like a list, tuple, or string) contains a specific element.

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

Operator	Name	Example	Description
<code>in</code>	In sequence	<code>x in my_list</code>	Returns <code>True</code> if the value is present in the sequence.
<code>not in</code>	Not in sequence	<code>x not in my_list</code>	Returns <code>True</code> if the value is not present in the sequence.

4. Operator Precedence

Precedence determines the order in which operators are evaluated in an expression. Just like in mathematics, multiplication and division are performed before addition and subtraction.

Order	Operator Group	Example
1. (Highest)	Parentheses	<code>(10+2)*3</code>
2.	Exponentiation	<code>2**3</code>
3.	Multiplication, Division, Modulus, Floor Division	<code>5*2/4</code>
4.	Addition, Subtraction	<code>10+5</code>
5. (Lowest)	Comparison, Identity, Membership, Logical (and, or)	<code>x>5 and x<10</code>

Tip: When in doubt about precedence, **always use parentheses ()** to make the evaluation order explicit and improve code readability.

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

The Python `math` Module

For any math beyond basic arithmetic, you need to use the standard `math` module. This module provides access to advanced mathematical functions and constants defined by the C standard.

How to Use the `math` Module

You must explicitly import the module before using it:

```
import math
# Now you can access its functions using the dot notation: math.function_name()
```

Key Functions and Constants in `math`

Category	Function/Constant	Description	Example
Constants	<code>math.pi</code>	The mathematical constant (Pi).	<code>math.pi</code>
	<code>math.e</code>	The mathematical constant e (Euler's number).	<code>math.e</code>
Power/Roots	<code>math.sqrt(x)</code>	Returns the square root of x.	<code>math.sqrt(25)</code> -> 5.0

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

	<code>math.pow(x, y)</code>	Returns x raised to the power of y.	<code>math.pow(2, 4)</code> -> 16.0
Rounding	<code>math.ceil(x)</code>	Returns the smallest integer greater than or equal to x (ceiling).	<code>math.ceil(4.1)</code> -> 5
	<code>math.floor(x)</code>	Returns the largest integer less than or equal to x (floor).	<code>math.floor(4.9)</code> -> 4
Trigonometry	<code>math.sin(x),</code> <code>math.cos(x),</code> <code>math.tan(x)</code>	Standard trigonometric functions (takes radians as input).	<code>math.sin(math.pi / 2)</code>

4. Controlling Precision and Rounding

Python offers several ways to handle decimal precision, depending on your goal.

A. Standard `round()` (Built-in Function)

The built-in `round()` function is used to round a float to the nearest integer, or to a specified number of decimal places.

```
# Round to nearest integer (Standard rounding)
result_1 = round(1.6) # Output: 2
```

```
# Round to a specific number of decimal places
result_2 = round(3.14159, 2) # Output: 3.14
```

A Note on `round()`: Python's `round()` uses "round half to even" (or banker's rounding) for numbers exactly halfway between two integers (e.g., `round(2.5)` is 2, while `round(3.5)` is 4). For beginners, this subtlety is often secondary to understanding the main use case.

B. Forced Rounding with `math.ceil()` and `math.floor()`

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)

If you need absolute rounding up or down, the `math` module functions are necessary:

Scenario (Ceiling): A customer orders 4.1 units of product. Since you can only ship whole units, you must round **up** to 5.

```
import math
```

```
units = math.ceil(4.1) # units is 5
```

•

Scenario (Floor): You calculate your profit ratio as 0.687. For a conservative estimate, you want to round **down** to 0.

```
import math
```

```
ratio = math.floor(0.687) # ratio is 0
```

Connect with Dr. Amar Panchal :17 Years of Expertise in Technical Coding & Placement Training (Co-founder, Career Credentials)
amarpanchal.in Connect: [\[LinkedIn\]](#) | [\[Instagram\]](#) | [\[Facebook\]](#) | [\[Youtube\]](#)